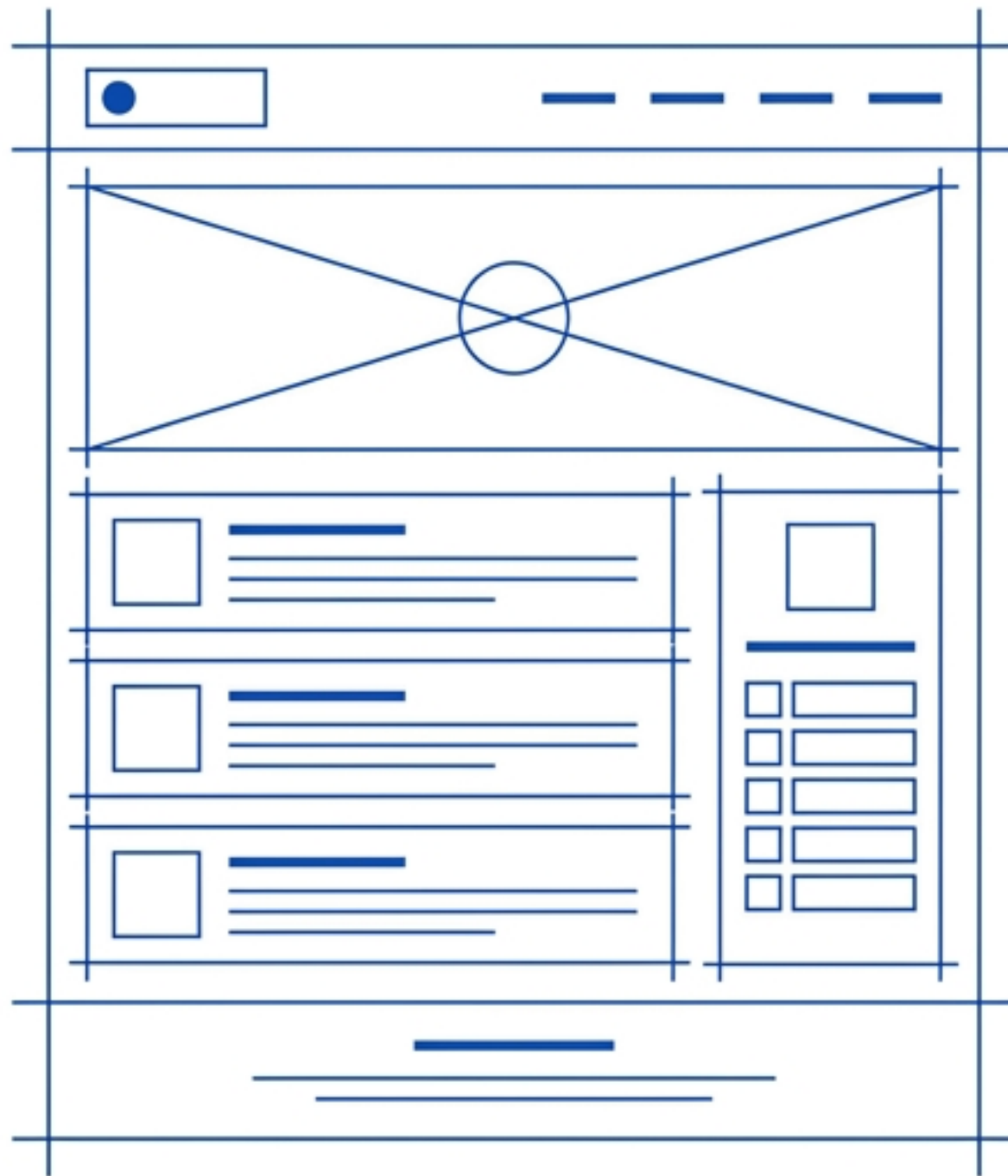




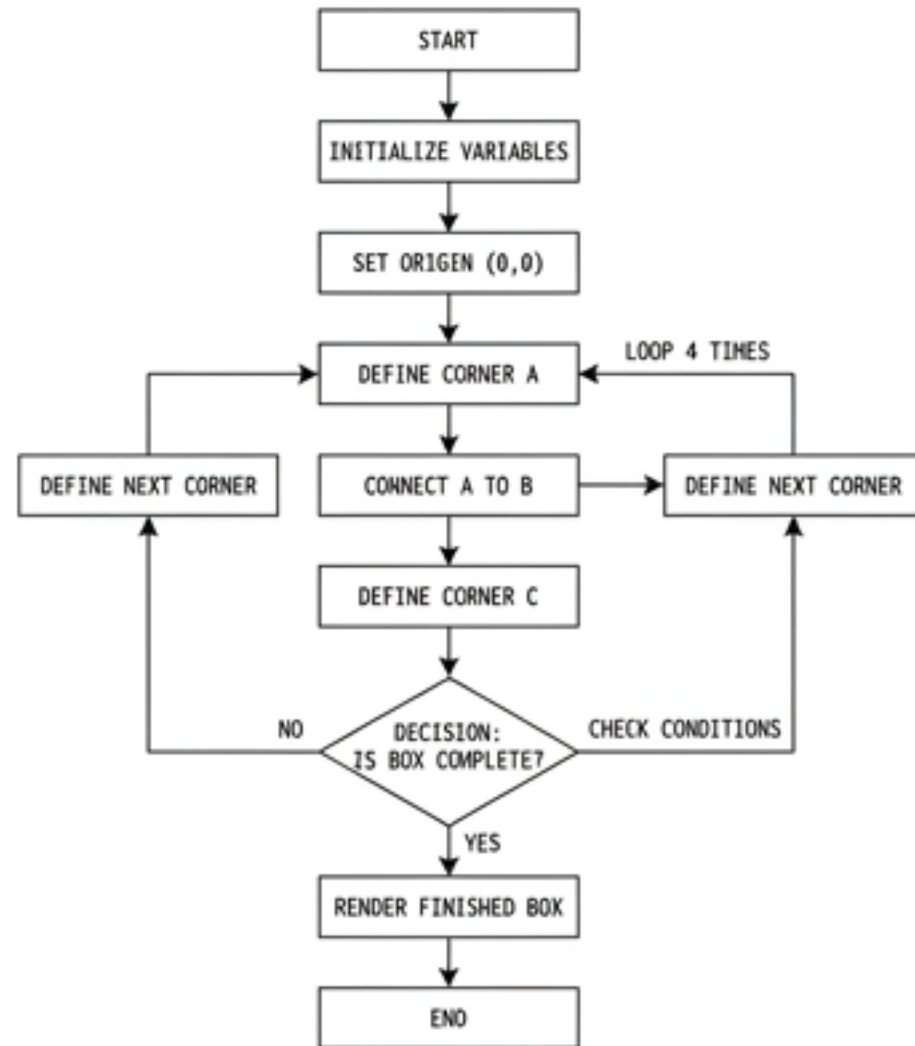
Arquitectura Web Estructurada

HTML, Etiquetas, Semántica y Código Limpio

Un enfoque desde el Paradigma Declarativo.

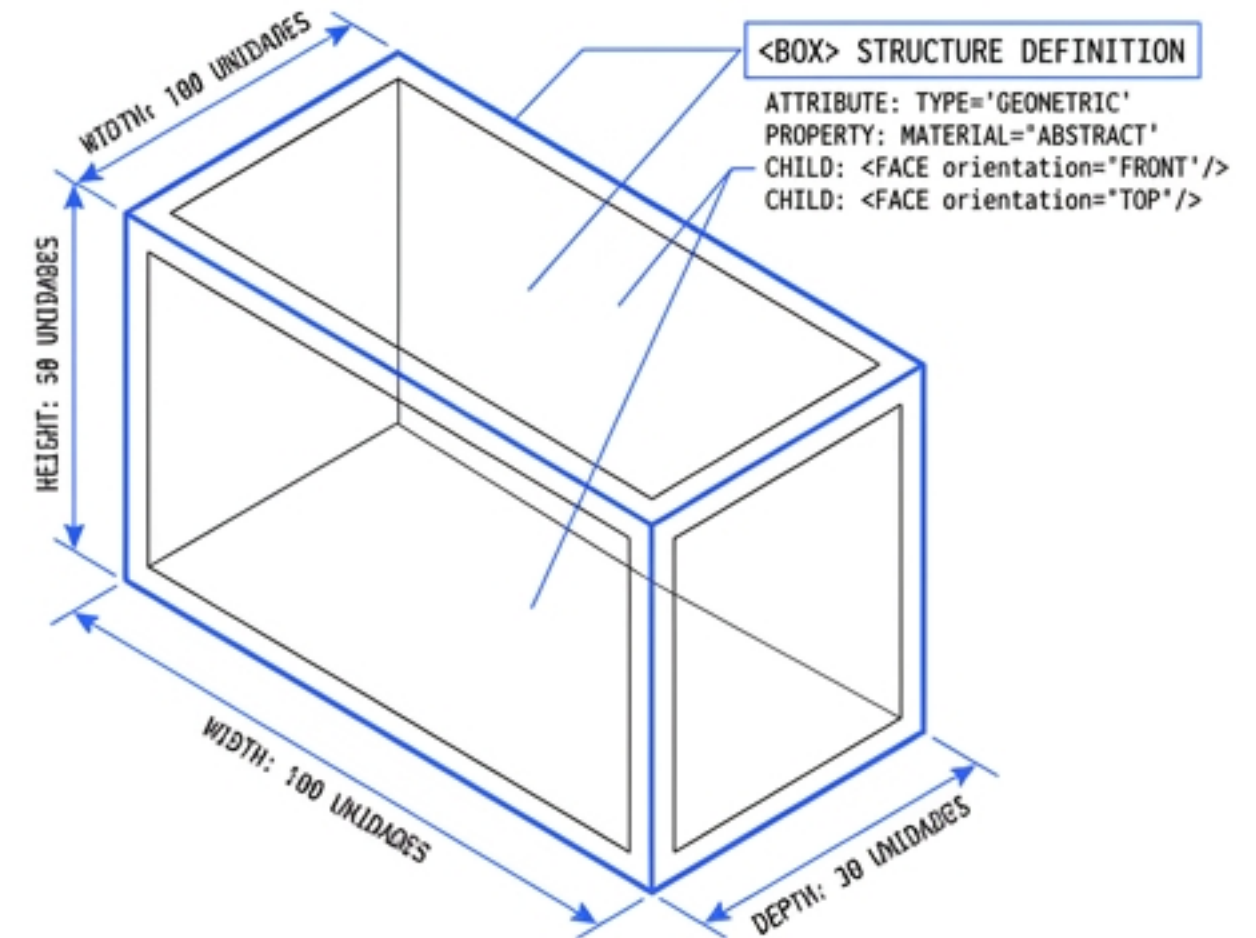
HTML en el Paradigma Declarativo

Enfoque tradicional (Imperativo)



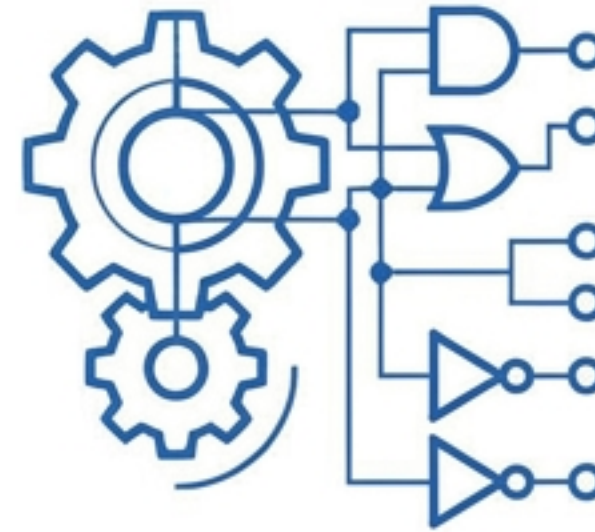
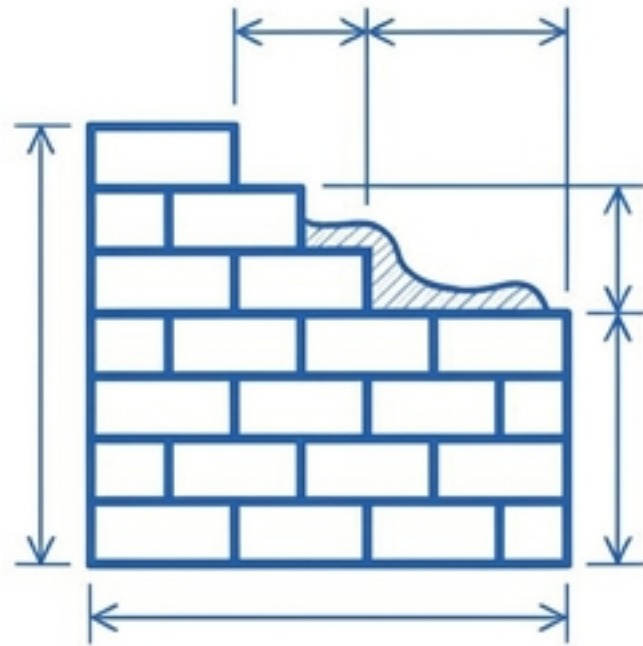
Describir paso a paso cómo lograr un resultado (ej. C, Pascal).

Enfoque de marcado (Declarativo)



Describir qué es la información y cómo está estructurada, delegando la presentación al navegador.

El lenguaje de la estructura, no del comportamiento



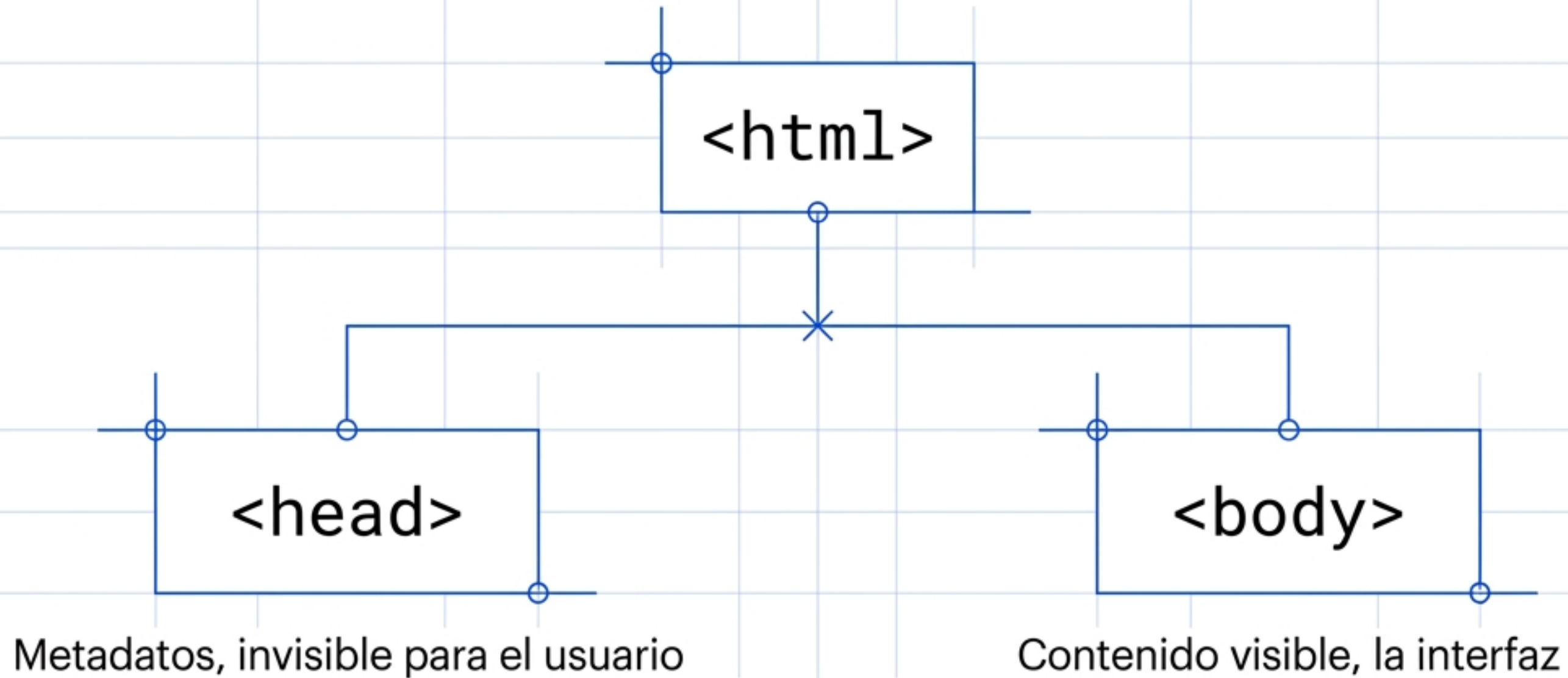
HTML (HyperText Markup Language) no es un lenguaje de programación. Carece de variables, bucles o lógica condicional. Su único propósito es organizar, clasificar y dar significado a la información mediante etiquetas.

Anatomía de una etiqueta



El esqueleto del documento

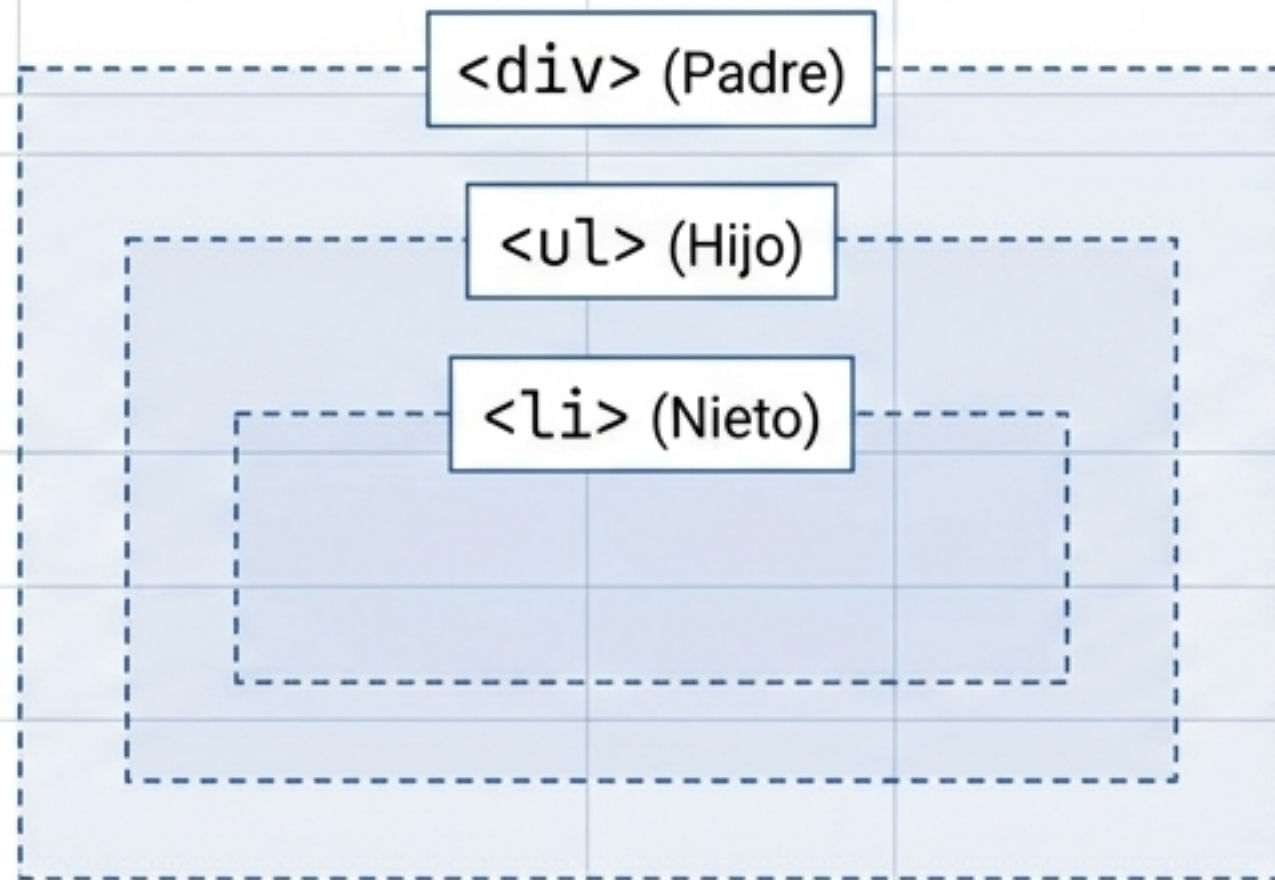
Todo documento requiere una estructura base innegociable. Separar los metadatos del contenido es el primer paso hacia un código limpio.



El principio de anidamiento

Las etiquetas contienen otras etiquetas creando una jerarquía estricta. Un bloque hijo debe cerrarse siempre antes de que se cierre su bloque padre.

Visual



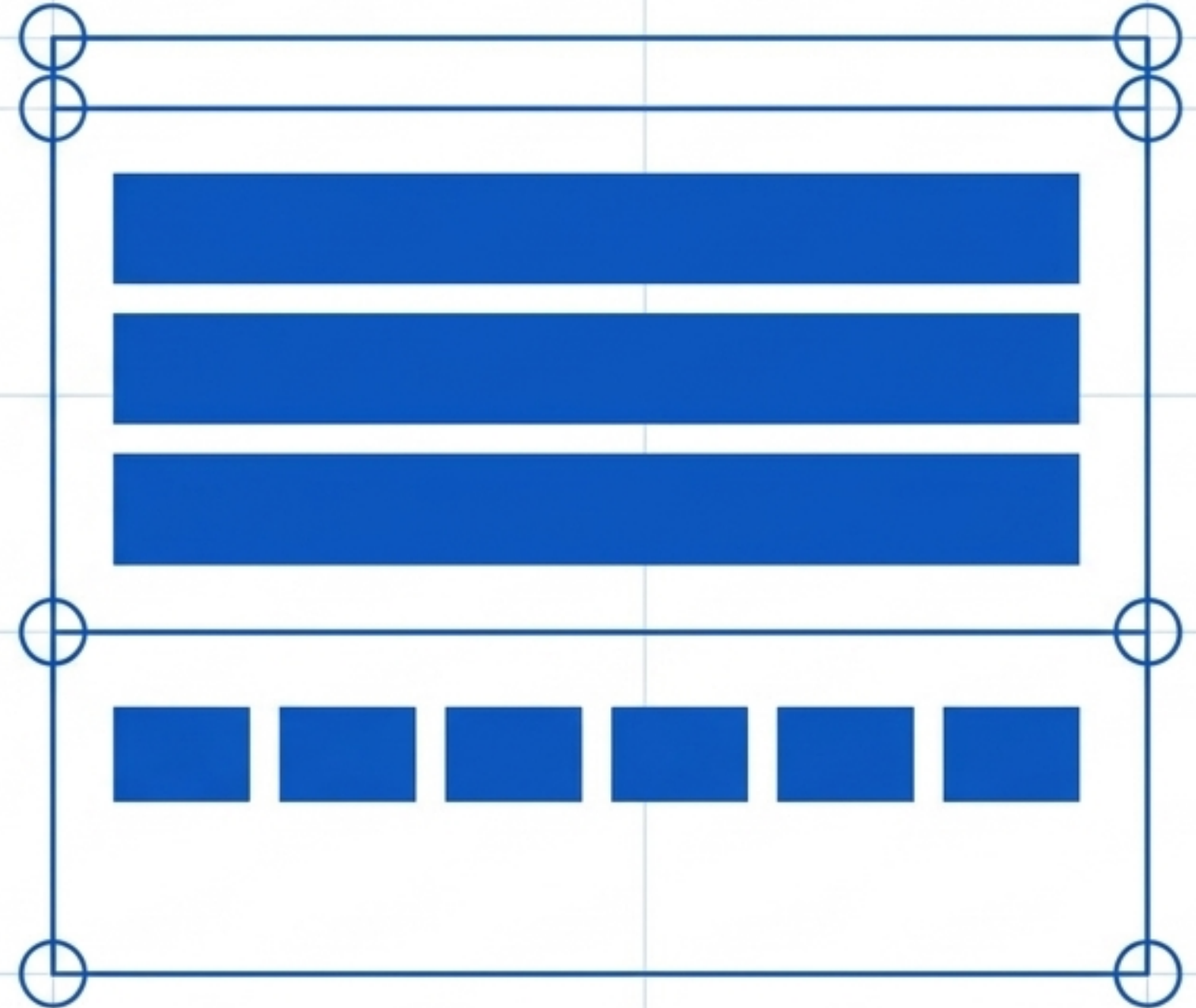
Syntax

```
<div>
|
|  <ul>
|  |
|  |  <li>...</li>
|  |
|  </ul>
|
</div>
```

Comportamiento espacial: Bloque vs. Línea

Bloque (<h1> , <p>):
Reclaman todo el ancho disponible. Fuerzan un salto de línea.

Línea (<a> ,):
Ocupan solo el espacio de su contenido. Conviven en la misma línea.



Jerarquía y formato de texto

Los niveles de encabezado estructuran la importancia de la información, no solo su tamaño visual.

<h1> Encabezado Principal

<h2> Sección Secundaria

<h3> Subsección

<p> Párrafo de texto base que contiene la información descriptiva del documento.

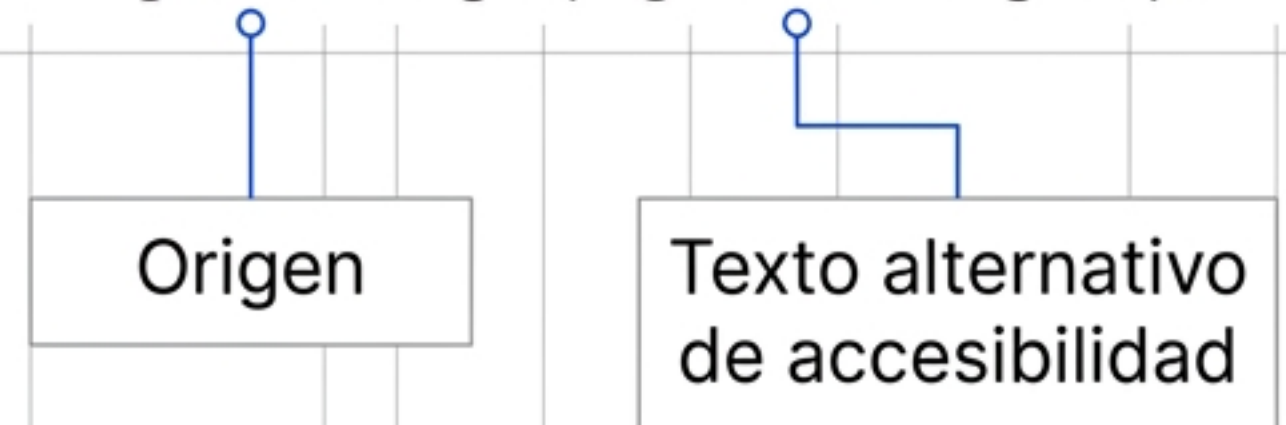
Enlaces y recursos externos

Las etiquetas `<a>` e `<img>` dependen de atributos específicos para funcionar, conectando el documento con el resto de la web.

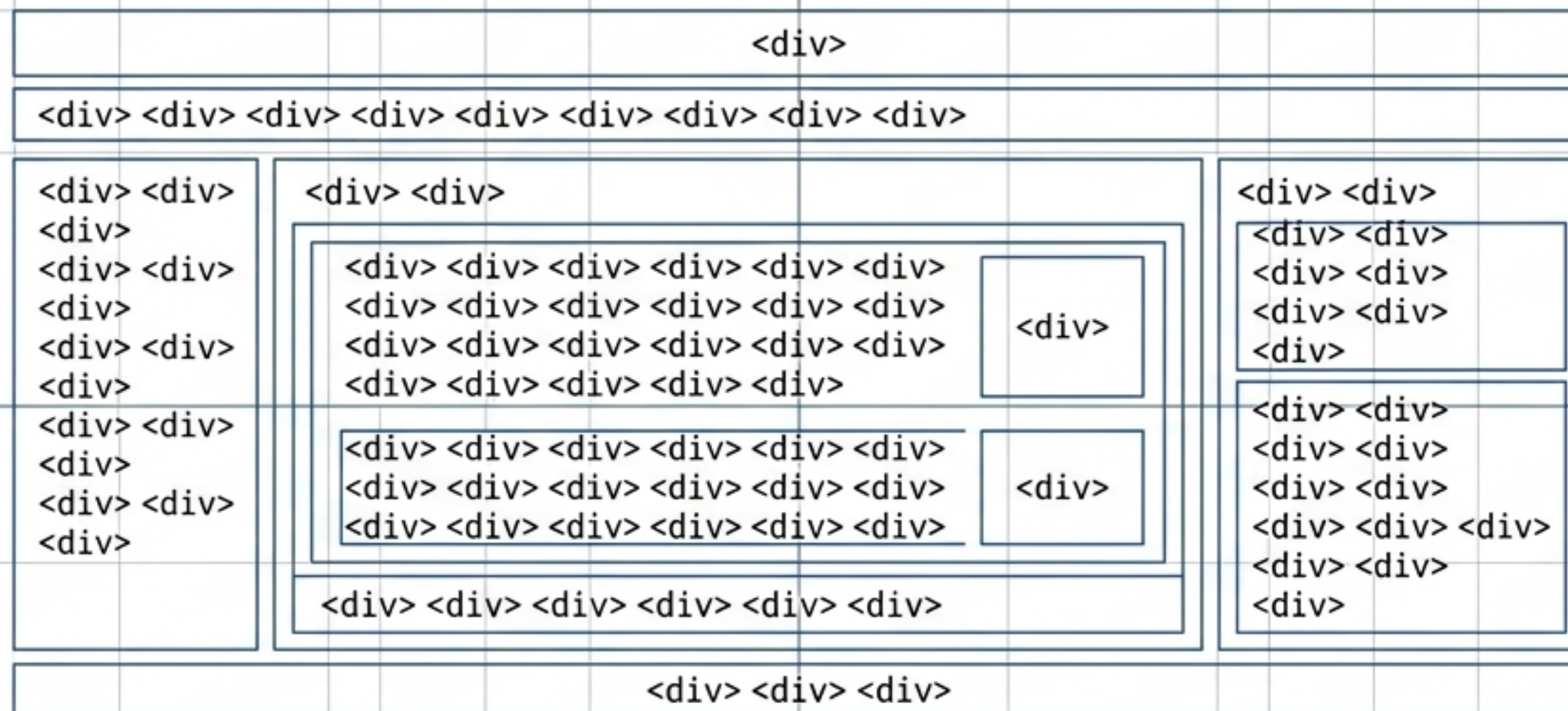
`<a href="https://unju.edu.ar">Sitio UNJu</a>`



``



El problema: La Deuda Técnica ("Sopa de Divs")



Un código puede funcionar visualmente, pero ser incomprensible a nivel estructural. Usar la etiqueta genérica `<div>` para todo genera deuda técnica, creando un documento imposible de mantener o escalar por otros programadores.

La solución: Nombres con sentido

La semántica en HTML aplica el principio de Clean Code de "Nombres con sentido". Las etiquetas deben describir su propósito y el tipo de información que contienen, no su apariencia.

`<div>` + Significado = Etiqueta Semántica

`<div id="encabezado">`



`<header>`

Matriz Comparativa: Genérico vs. Semántico

Código Sucio ✕

Código Limpio ✓

```
<div class="header">  
  <div id="nav">  
  <div class="footer">
```

```
<header>  
  <nav>  
  <footer>
```

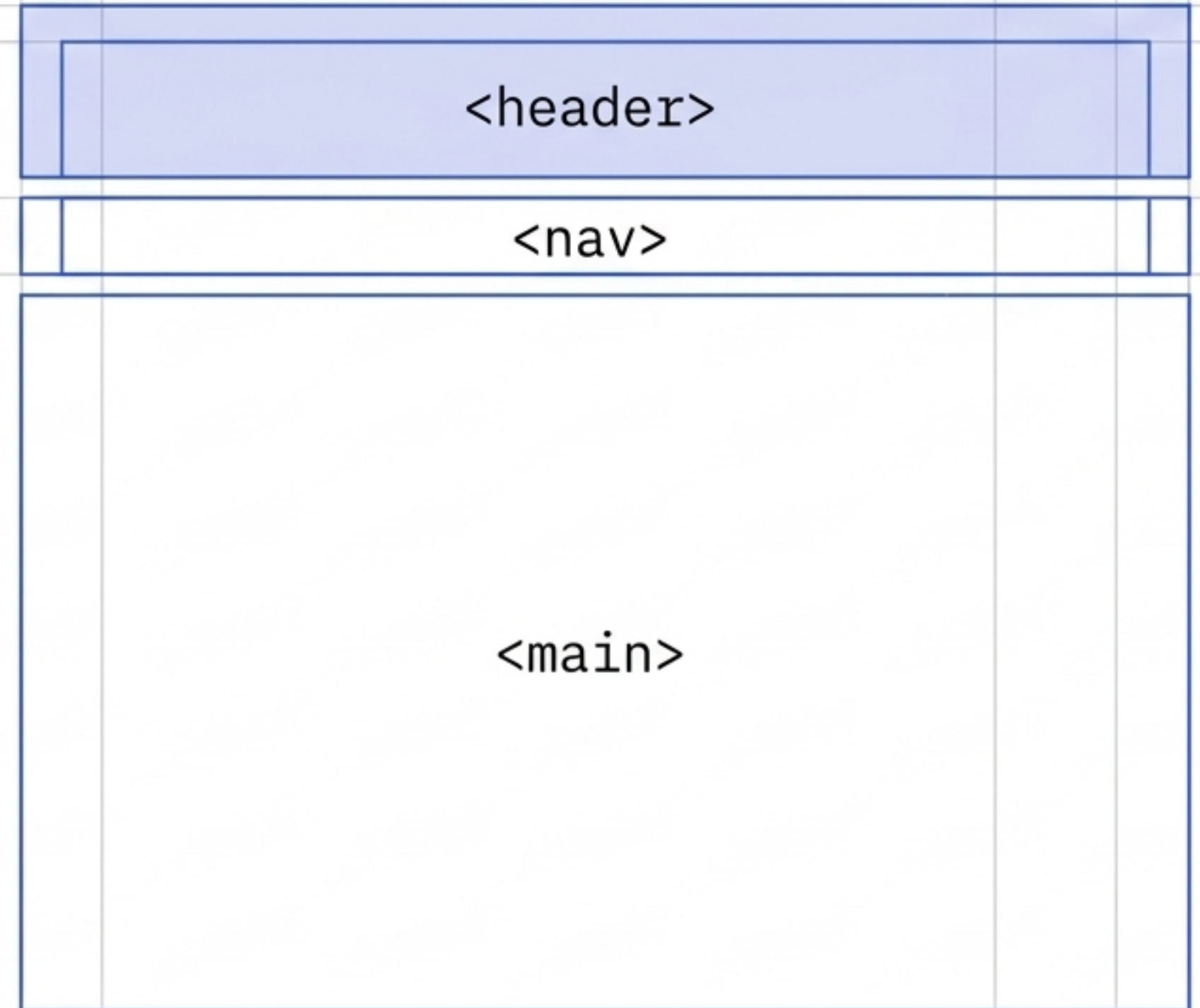
- Legibilidad: El código semántico es autodescriptivo.
- Mantenibilidad: Menos dependencia de clases (IDs) artificiales.
- Accesibilidad: Los lectores de pantalla entienden la estructura nativamente.

Semántica estructural: Contenedores principales

`<header>`: Introducción o cabecera del sitio/sección.

`<nav>`: Bloque principal de enlaces de navegación.

`<main>`: El contenido central y único del documento.

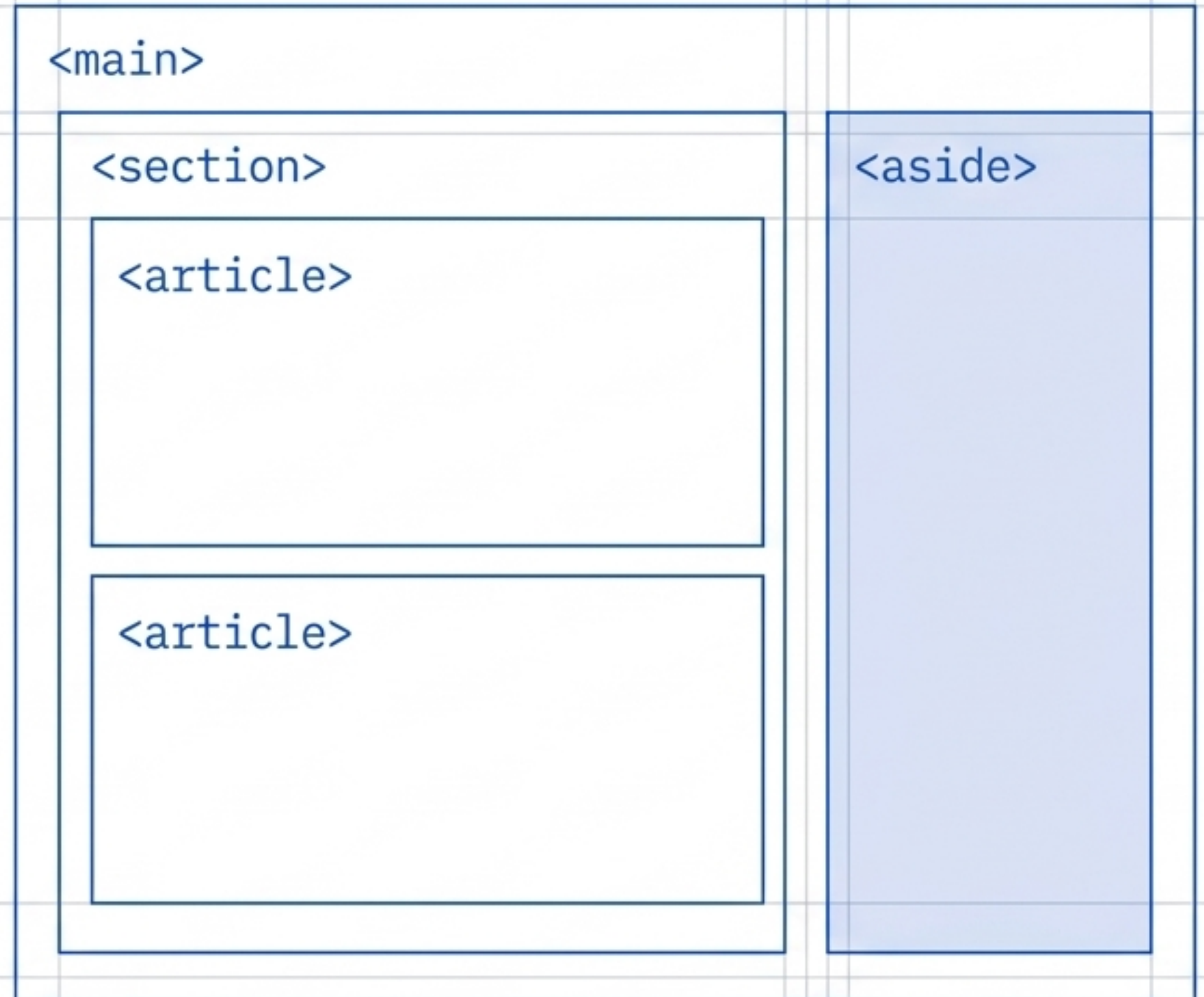


Semántica estructural: División de contenido

`<section>`: Agrupación temática de contenido.

`<article>`: Contenido independiente y autoconclusivo (ej. una noticia).

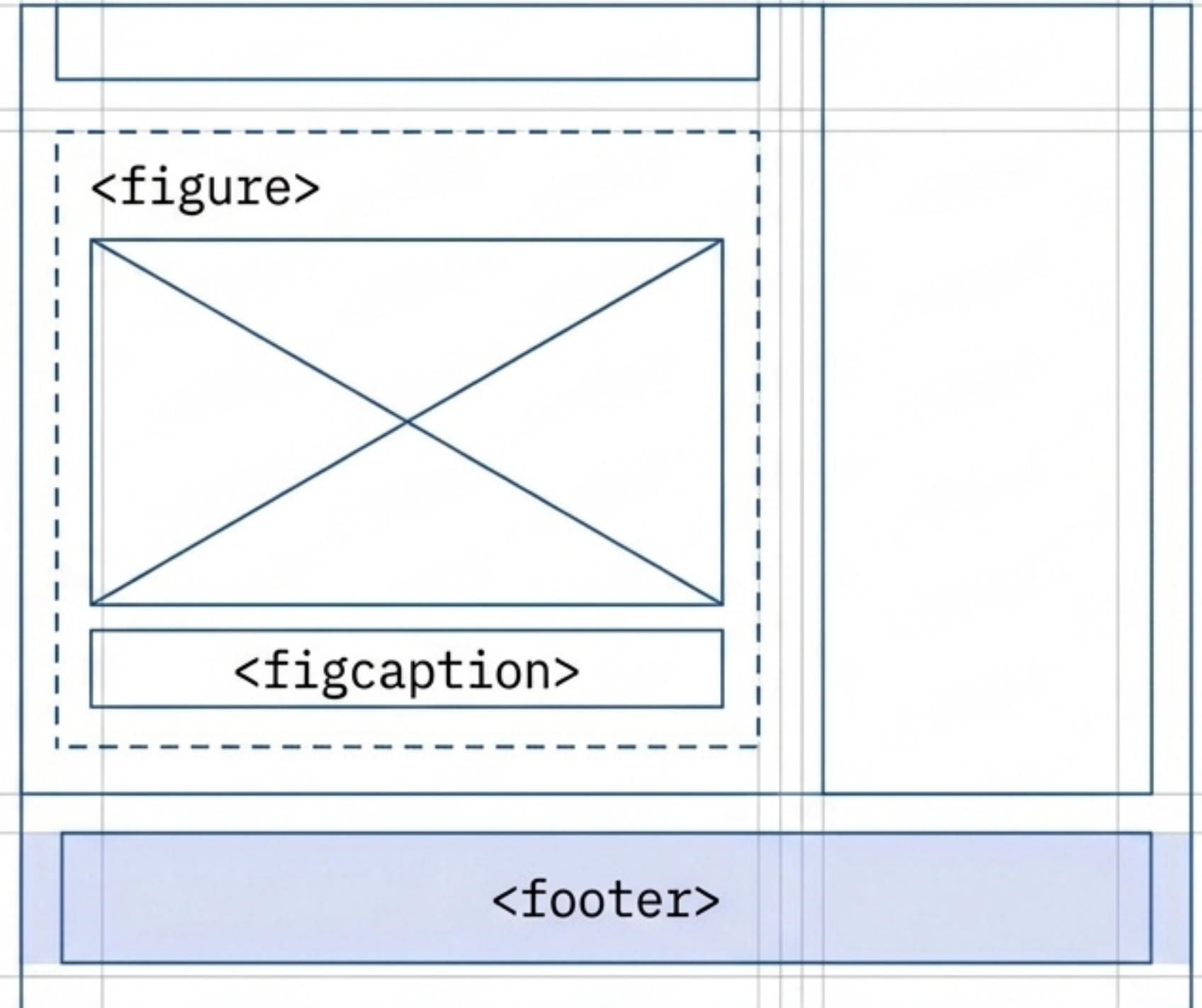
`<aside>`: Contenido tangencial o barra lateral.



Semántica estructural: Detalles y cierre

`<figure>` y `<figcaption>`:
Encapsulan medios gráficos con su
respectiva leyenda contextual.

`<footer>`: El cierre del documento o
sección (copyright, enlaces legales).



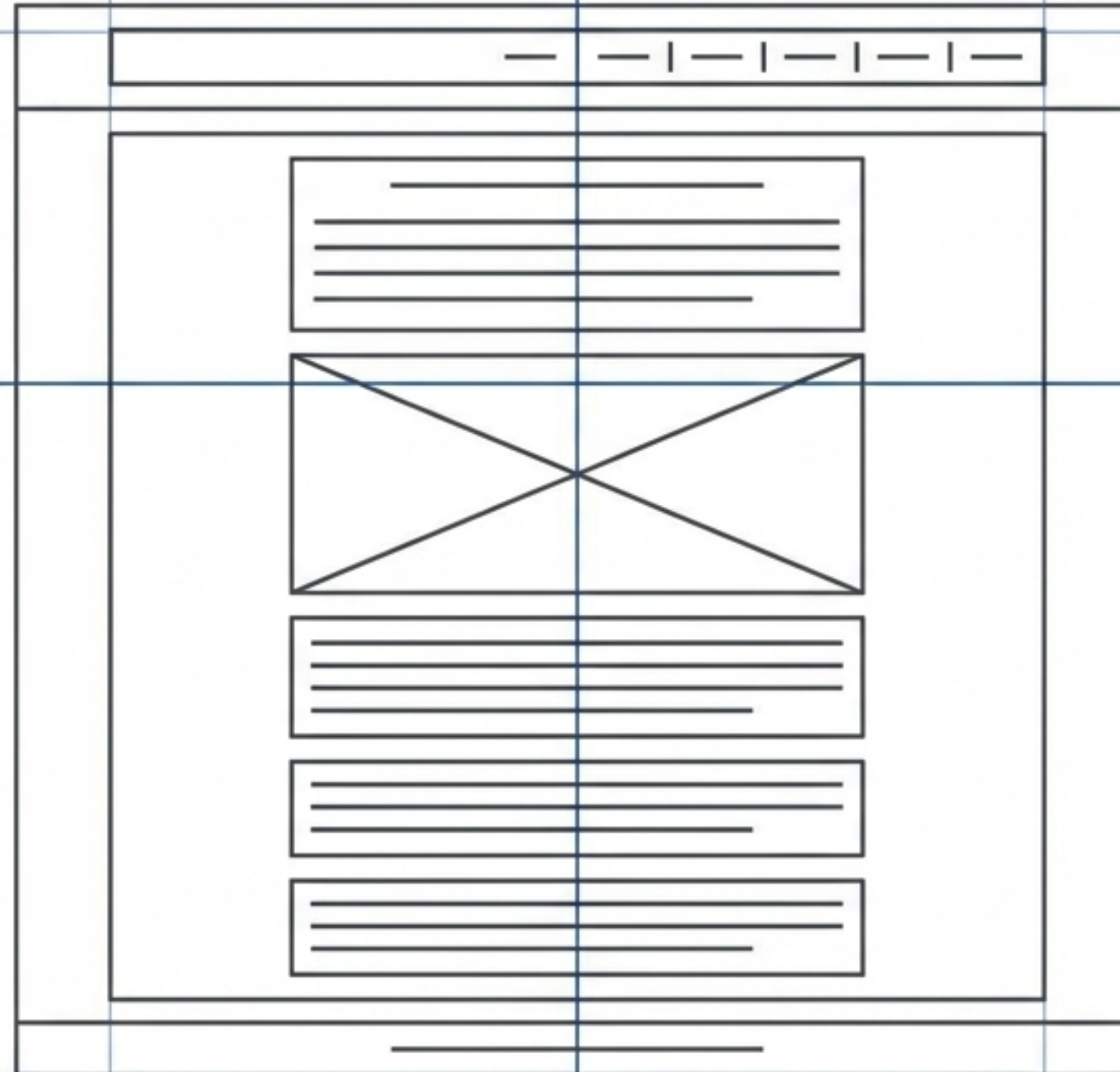
Caso Práctico: Diseño de un Blog

Requerimiento: Diseñar el esqueleto HTML para un artículo de blog.

Objetivo: Aplicar el principio de "Nombres con sentido" para evitar la deuda técnica, transformando un diseño visual en una arquitectura de código limpio.

El Plano Visual (Wireframe)

Este es el diseño objetivo. El desafío arquitectónico es elegir los contenedores estructurales correctos para representar esta jerarquía.



El enfoque "Imprudente" (Sopa de Divs)

Este código logrará el resultado visual mediante CSS, pero sacrifica la semántica. Es un ejemplo clásico de deuda técnica: difícil de leer, incomprensible para motores de búsqueda y hostil para la accesibilidad.

```
<div class="top-bar">
  <div class="menu">...</div>
</div>
<div class="main-content">
  <div class="post">
    <div class="post-title">...</div>
    <div class="post-image">...</div>
    <div class="post-text">...</div>
  </div>
</div>
<div class="bottom-bar">...</div>
```

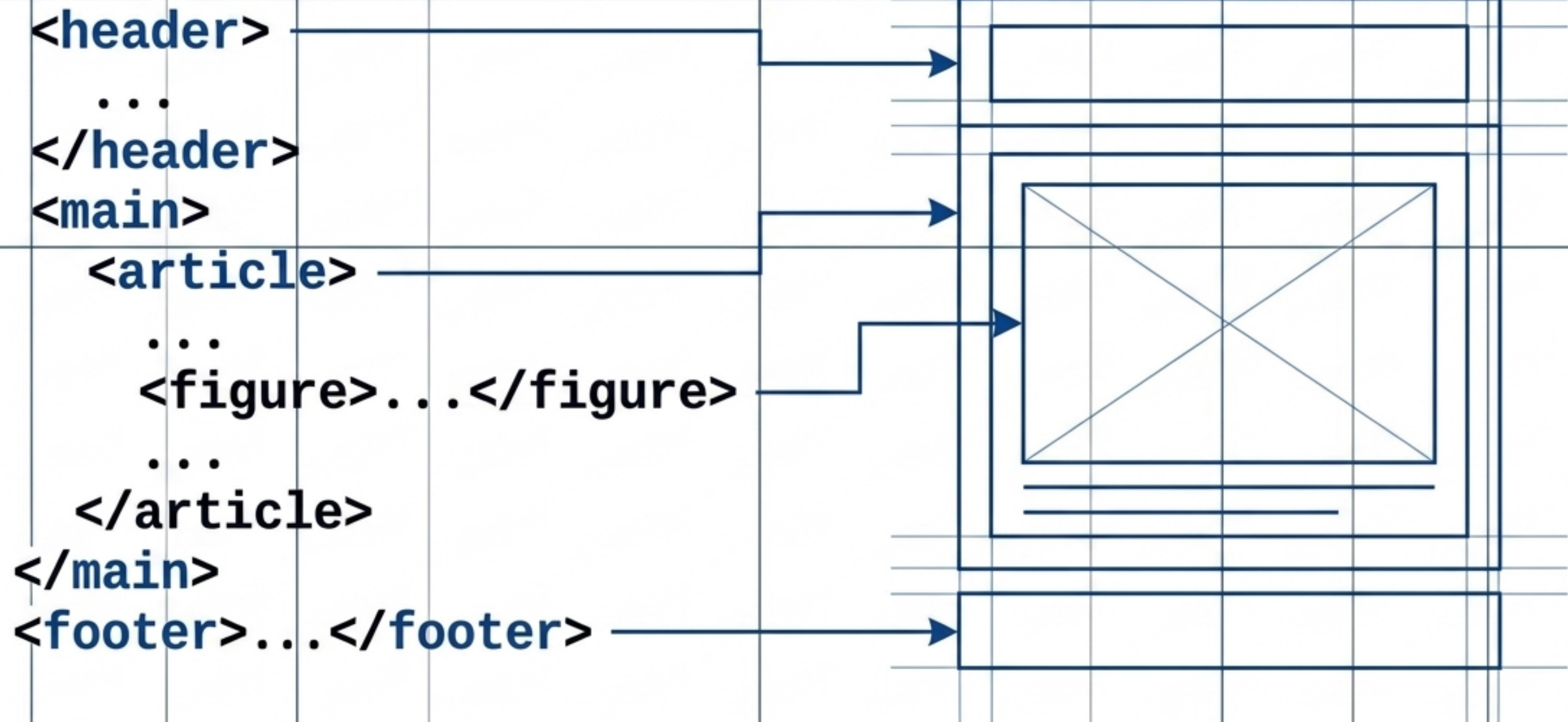
El enfoque "Prudente" (Código Limpio)

Refactorización semántica. El código ahora documenta su propia intención. Hemos eliminado la dependencia de clases artificiales para describir la estructura fundamental.

```
<header>  
  <nav>...</nav>  
</header>  
<main>  
  <article>  
    <h1>...</h1>  
    <figure>...</figure>  
    <p>...</p>  
  </article>  
</main>  
<footer>...</footer>
```

El mapeo estructural

La arquitectura semántica alinea perfectamente el modelo mental del programador (código) con la representación espacial (interfaz).



Conclusiones de Arquitectura



1. Mantenibilidad:
El HTML semántico reduce la deuda técnica y facilita la refactorización.



2. Accesibilidad:
Permite que tecnologías asistivas interpreten el documento correctamente.



3. Claridad (Clean Code):
Las etiquetas semánticas son "nombres con sentido" integrados en el lenguaje.